NaCRE: Native Confidential Containers with RISC-V Enforcement

Linke Song*†, Wenhao Wang*†✉, Weijie Liu‡, Rui Hou*†
*State Key Laboratory of Cyber Space Security, Defense Institute of Information Engineering, CAS, Beijing, China
†School of Cybersecurity, University of Chinese Academy of Sciences, Beijing, China
‡Nankai University

I. Motivation

Confidential containers seek to protect containerized workloads from privileged infrastructure while preserving the lightweight Linux container model. Existing systems have mainly reused hardware mechanisms designed for other execution units. Process-level TEEs, such as Intel SGX [1], [3], keep the protected domain small, but their process-oriented abstraction makes it difficult to support full container semantics, especially multi-process workloads and intra-container memory sharing. VM-level TEEs, such as AMD SEV [5], [6], Intel TDX [4], and ARM CCA [2], can host a familiar Linux stack, but typically protect each container or pod inside a microVM. In a preliminary x86-64 measurement using the same Alpine image and workload, Kata Containers incurs about 31.5× higher idle memory footprint and 2.34× higher end-to-end fork workload latency than Docker/runc.

Recent systems move the protection boundary closer to containers. BlackBox [8], RContainer [9], and Fasco [7] are valuable steps toward container-granularity protection, but they still rely on protection machinery originally designed around processes, VMs, or realm-like execution units. The missing piece is therefore not merely a choice between process-level and VM-level TEEs, but a native confidential-container substrate.

II. Requirements and Challenges

We believe a native confidential-container design should satisfy four requirements:

- **R1: Strong isolation.** Protect container memory, register state, and security metadata from an untrusted OS and other containers.
- **R2: Lightweight execution.** Keep startup, teardown, boundary crossing, and steady-state execution close to native containers.
- **R3: Linux-native semantics.** Support ordinary container process and memory behavior, including fork, clone, shared mappings, shared-memory IPC, and futex synchronization.
- **R4: Scalable utilization.** Avoid per-container VM-like duplication and static protected-memory reservation when they are not intrinsic to containers.

Satisfying these requirements together exposes three design challenges. **C1: strong isolation without heavyweight switching.** A confidential-container substrate must protect memory and security metadata from the host OS, but doing so by introducing monitor-managed shadow page tables, frequent page-table switches, or VM/Realm-style domain transitions would undermine the lightweight execution model of containers. This challenge becomes sharper when the design avoids reusing hardware mechanisms originally built for VMs or realms.

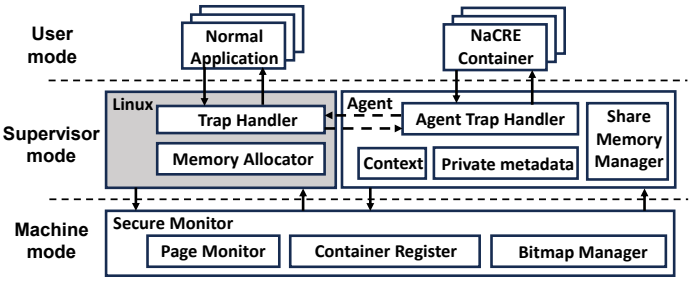

Fig. 1: NaCRE overview.

C2: safe handling of Linux-native accesses. A native container still executes through Linux. The kernel may legitimately touch user memory through paths such as user-copy helpers, signal delivery, page-fault handling, and file-system or IPC operations. These accesses are not necessarily malicious, but an untrusted kernel must not be allowed to read or corrupt confidential pages directly. The design must therefore distinguish allowed service operations from forbidden direct accesses without breaking ordinary Linux process semantics.

C3: isolation and sharing under Linux lifecycle management. The host OS and other containers must not access confidential pages, while mutually trusting processes inside the same container still need fork, clone, shared mappings, shared-memory IPC, and futex synchronization. At the same time, Linux should retain non-security-critical resource management where possible. The challenge is to let Linux manage the container lifecycle while preventing it from bypassing page ownership, sharing policy, and protected metadata.

III. NaCRE Design Overview

NaCRE addresses these challenges with a split enforcement model, as shown in Figure 1. A NaCRE container remains ordinary Linux processes rather than a guest VM or a single-process enclave. A trusted agent receives security-sensitive traps, protects private metadata, and validates operations before forwarding ordinary, non-security-critical service requests to Linux. A small secure monitor in trusted OpenSBI provides the enforcement root for ownership, metadata, and sharing policy: operations that change this protected state must request the monitor through an `ecall`. This organization lets Linux continue to provide common services, while the agent and

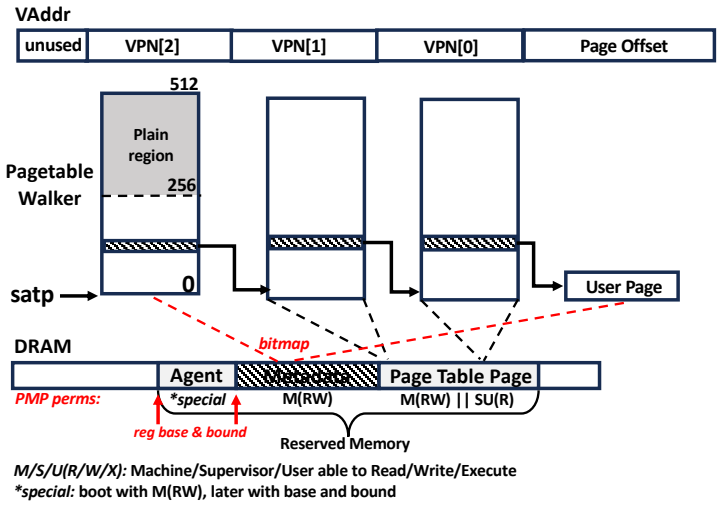

Fig. 2: NaCRE memory isolation.

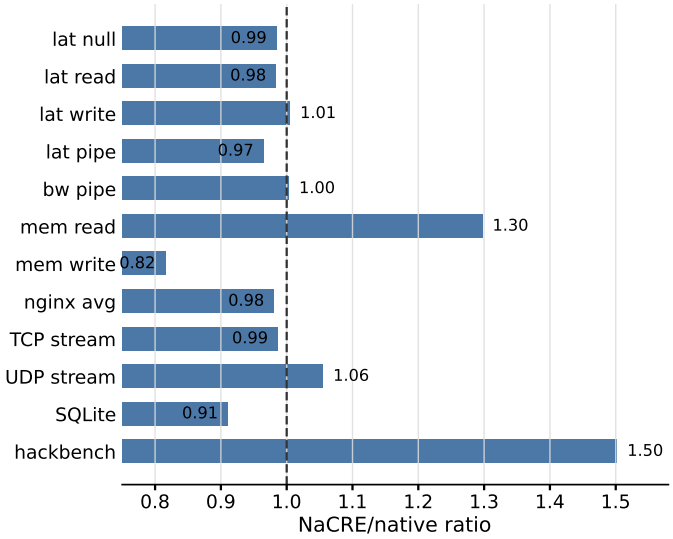

Fig. 3: Preliminary NaCRE/native ratios.

monitor guard the security state that Linux can no longer change unilaterally.

IV. Memory Isolation

The central mechanism behind this split is page-granular ownership enforcement. Figure 2 sketches how NaCRE separates ordinary memory management from security enforcement. Linux first allocates a physical page through its ordinary allocator, but this alone does not make the page confidential. Linux or the agent must issue an `ecall` to trusted OpenSBI, which validates the request and marks the page in the ownership bitmap. After this transition, the page leaves Linux's ordinary page lifecycle: Linux cannot directly reuse, remap, or alias it into the host or another container.

NaCRE also protects the translation structures that make these pages reachable. The agent state and ownership bitmap reside in the metadata region, and the bitmap records page-granular ownership for the root pagetable page referenced by `satp` and for confidential user pages. The pagetable pages themselves are protected differently: their contents are placed under PMP-style permissions, which make them readable for MMU walks but not writable by the untrusted kernel. During translation checks, NaCRE follows the RISC-V Linux high-half convention: the high-half kernel portion of the root page table is allowed to pass through, while low-half user mappings are checked against protected-page ownership.

V. Preliminary Results

Figure 3 reports early ratios from our current prototype on a single-core emulated RISC-V setup hosted on an x86_64 AMD EPYC 7503 server with modified QEMU 9.2.0 and Linux 6.12.0. The tests cover different container-facing paths: lmbench probes syscall latency plus pipe and memory bandwidth, nginx and netperf exercise service throughput and network paths, SQLite checks an application-level storage workload with identical output hashes, and hackbench stresses thread-heavy message passing. Overall, NaCRE stays close to native on syscall, pipe, nginx (0.981× average throughput), netperf, and SQLite (0.91× time in this run). The main visible gap is hackbench, whose internal runtime is 1.50× on a workload that creates 2,000 pthread workers, pointing to process/thread setup and serving-path serialization rather than a broad steady-state slowdown.

VI. Next Steps

The preliminary results point to both lifecycle and scalability work. Next, we will evaluate Docker lifecycle commands such as `docker create`, `docker pause`/`unpause`, and `docker kill`, since they expose container-management overhead beyond in-container steady-state execution. We will also parallelize the currently serialized serving path, stress-test many concurrent confidential containers, add cross-container and finer intra-container sharing, and measure Linux/agent switches, OpenSBI `ecall` paths, and custom enforcement instructions on real RISC-V hardware.

References

[1] "Intel software guard extensions overview," https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html. Referenced December 2022, 2022.

[2] Arm, "Arm Confidential Compute Architecture," https://www.arm.com/architecture/security-features/arm-confidential-compute-architecture, 2023.

[3] Intel, "Intel software guard extensions developer guide," https://www.intel.com/content/www/us/en/content-details/671334/intel-software-guard-extensions-intel-sgx-developer-guide.html. Referenced December 2022.

[4] ——, "Intel Trust Domain Extensions," https://software.intel.com/content/dam/develop/external/us/en/documents/tdxwhitepaper-v4.pdf, 2020.

[5] D. Kaplan, J. Powell, and T. Woller, "Amd memory encryption," *White paper*, 2016.

[6] A. SEV-SNP, "Strengthening vm isolation with integrity protection and more," *White Paper, January*, 2020.

[7] L. Song, Y. Zhang, F. Zhang, Y. Ding, B. Zhou, J. Yu, and Y. Tan, "Shielded but lightweight: Building practical confidential containers with ARM CCA," arXiv preprint arXiv:2605.26018, 2026.

[8] A. Van't Hof and J. Nieh, "BlackBox: a container security monitor for protecting containers on untrusted operating systems," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 683–700.

[9] Q. Zhou, W. Cao, X. Jia, P. Liu, S. Zhang, J. Chen, S. Xu, and Z. Song, "Rcontainer: A secure container architecture through extending arm cca hardware primitives." NDSS, 2025.